Starter Page 2

The problem

Monitoring CVA (credit valuation adjustment) by counterparty. The deliverable is a small dashboard: one filterable table of CVA charge per counterparty, backed by 20 mock rows (counterparty, CDS spread, tenor, CVA charge, timestamp). CDS spreads and CVA charges are static mock inputs, not computed — this is a monitoring UI, not a pricing engine.

The project is a vehicle to use one AI tool per SDLC stage and record why. It draws on real CVA domain depth: a full-stack CVA UI was previously built in React/C#, and rebuilding a deliberately small version of that surface through this pipeline shows the upskilling via AI-assisted development. Target aesthetic: Bloomberg/AG Grid terminal density, not consumer SaaS.

The approach — one AI tool per SDLC stage

Stage	Tool	Why this tool	What went wrong or taught me
Design	Stitch	To generate dense, terminal-style designs fast.	Figma Make/AI Studio (compared) drifted on typography and density; light variant is the build target, dark is reference only.
Build	Replit Agent	Spec-to-code with a controlled structure.	“No backend” was ignored at the scaffold level (pnpm monorepo, unused Express api-server, Postgres/Drizzle, OpenAPI); the unused scaffold was left in place, not pruned.
Component testing	Storybook	To isolate `CvaGrid`.	`storybook init` over-installed and was pruned back to two packages; a green build did not prove the story rendered — screenshots are the evidence.

Stage	Tool	Why this tool	What went wrong or taught me
E2E testing	Playwright	For the user-visible behaviour: the filter narrows the table.	My hand count (3 rows) was wrong — “Citigroup” contains “group”; expectations should come from the data.
Review	CodeRabbit	To review a planted edge in PR #21.	Installed, but never responded; cause unconfirmed.
Docs	Notion AI	To turn the project notes into a concise one-page.	Done

The AI-assisted workflow taught me to name exactly what to change.

The stack

React + TypeScript, Vite, Tailwind, Storybook, Playwright. Data: `design/mock-data.json`. Runs locally with plain npm; no backend.

What I'd change next time

- Check each prompt against recorded decisions before running it.
- Prune or constrain tool defaults early — scaffolds and generators over-deliver.
- Verify third-party integrations on a throwaway PR first — installs can fail silently.
- Derive test expectations from the data, and prove tests can fail.
- Open the rendered output; a green build is not evidence.

Repo: https://github.com/bleunguts/claude-ai-sdlc-cva-dashboard

README.md: design/README.md
CvaGrid.stories: CvaGrid.stories.tsx

End2End test: e2e/cva-filter.spec.ts

PR #21: https://github.com/bleunguts/claude-ai-sdlc-cva-dashboard/pull/21